IP Centrul de Excelenta in Informatica
si Tehnologii Informationale (CEITI)

Catedra de Informatica I

Studiul Individual Nr. 1

Disciplina: **Programarea vizuala**

Tema:

Dezvoltarea jocului desktop
Zombie Survivors in C# Windows Forms -
Proiectarea interfetei grafice
si implementarea functionalitatilor

Elev: Bragari Eduard

Grupa: P-2432

Profesor: Diana Nebunealia

Chisinau, 2026

Cuprins

1. Documentare teoretica 4
1.1 Ce este programarea vizuala? 4
1.2 Ce este Windows Forms? 4
1.3 Limbajul C# si platforma .NET 4
1.4 Cerinte functionale 5
1.5 Cerinte non-functionale 5
1.6 Tabel comparativ 5

2. Prezentarea aplicatiei 6
2.1 Descrierea jocului 6
2.2 Functionalitati principale 6
2.3 Tehnologii utilizate 6
2.4 Captura generala a jocului 6

3. Structura proiectului 7
3.1 Organizarea fisierelor 7
3.2 Rolul formularului Form1 7
3.3 Legatura dintre componente 7

4. Realizarea practica a interfetei 8
4.1 Crearea proiectului Windows Forms 8
4.2 Utilizarea Toolbox-ului 9
4.3 Form Designer si Properties 10
4.4 Crearea meniului principal 12
4.5 Crearea barei HP 13
4.6 Bara XP si nivelul 14
4.7 Interfata pentru valuri 15
4.8 Interfata Level Up 17
4.9 Pause Menu 20
4.10 Settings 21
4.11 Game Over si Respawn 22
4.12 Debug Menu 23

4.13 Legatura dintre UI si cod 24

5. Explicarea codului jocului 27
5.1 Miscarea jucatorului 27
5.2 Sistemul de shooting 28
5.3 Inamicii 29
5.4 Sistemul de valuri 29
5.5 XP si Level Up 30
5.6 Upgrade-urile 30
5.7 Sistemul HP si bandajele 30
5.8 Randarea grafica 30

6. Reflectie personala 31

7. Git si GitHub 31
7.1 Ce sunt Git si GitHub? 31
7.2 Utilizarea Git in proiect 32
7.3 Repository-ul proiectului 32

8. README si alte comenzi Git 32
8.1 Ce este README? 32
8.2 Comenzi Git utile 32

9. Concluzie 33

10. Bibliografie 34

1. Documentare teoretica

1.1 Ce este programarea vizuala?

Programarea vizuala permite construirea interfetei unei aplicatii prin elemente grafice: ferestre, butoane, liste si panouri. In Visual Studio, pot aseza un control pe formular si ii pot modifica dimensiunea sau culoarea fara sa scriu manual fiecare instructiune de configurare. Designer-ul salveaza aceste alegeri in cod C#. [1, 2]

Configurarea vizuala stabileste *cum arata* interfata. Codul scris de programator stabileste *ce se intampla* cand utilizatorul interactioneaza cu ea. De exemplu, butonul PLAY poate fi desenat si pozitionat in Designer, dar pornirea unei runde este realizata de metoda C# asociata evenimentului Click.

Acest mod de lucru se foloseste la aplicatii de evidenta, formulare de inscriere, calculatoare, editoare si proiecte educationale. In Zombie Survivors, aceeasi abordare este aplicata meniurilor si indicatoarelor jocului.

1.2 Ce este Windows Forms?

Windows Forms este o tehnologie .NET pentru aplicatii desktop pe Windows. O fereastra este reprezentata printr-un obiect Form, iar in interiorul ei pot exista controale precum Button, Label, Panel sau ComboBox. Controalele expun proprietati, metode si evenimente. Pentru desene proprii, formularul ofera acces la obiectul Graphics. [1]

Pentru proiectul meu, avantajul principal este ca pot vedea imediat structura interfetei si pot conecta fiecare buton la o metoda. Folosesc controale standard pentru meniuri si HUD, iar arena si personajele sunt desenate separat. Astfel, pot exersa atat programarea interfetei, cat si organizarea logicii in clase.

1.3 Limbajul C# si platforma .NET

C# este limbajul in care este scris proiectul, iar .NET ofera mediul de executie si bibliotecile folosite. Fisierul proiectului declara tinta **net10.0-windows** si optiunea **UseWindowsForms**. Aceasta este configuratia reala analizata, nu o aplicatie .NET Framework mai veche. [3, 4]

Notiune	**Exemplu concret in proiect**
Clasa si obiect	Player descrie un jucator; session.Player este obiectul folosit in runda.
Variabila / proprietate	Health pastreaza viata curenta, iar Level pastreaza nivelul.
Metoda	Reset() reinitializeaza sesiunea; UpdateHealth() actualizeaza HUD-ul.
Eveniment	Click anunta apasarea unui buton; Tick declanseaza actualizarea periodica.

Cerintele aplicatiei

1.4 Cerinte functionale

O cerinta functionala descrie o actiune pe care aplicatia trebuie sa o realizeze. In acest proiect, jucatorul se deplaseaza cu WASD, tinteste cu mouse-ul si trage cu butonul stang. Inamicii apar in valuri, urmaresc jucatorul si ofera XP atunci cand sunt eliminati. La cresterea nivelului se afiseaza trei optiuni de upgrade.

Interfata trebuie sa permita pornirea jocului, pauza, alegerea modului de afisare si reluarea unei runde dupa Game Over. Bara HP, progresul XP, nivelul si numarul inamicilor trebuie sa reflecte datele curente ale sesiunii.

1.5 Cerinte non-functionale

O cerinta non-functionala descrie calitatea utilizarii: claritate, stabilitate, performanta sau compatibilitate. Pentru un joc cu multe actualizari, interfata trebuie sa ramana lizibila, iar obiectele temporare trebuie eliminate cand nu mai sunt necesare. Compatibilitatea vizata este Windows, conform configuratiei proiectului.

Timer-ul are intervalul configurat la 16 ms, adica un ritm nominal de aproximativ 62,5 actualizari pe secunda. Aceasta setare nu demonstreaza un numar garantat de cadre: incarcarea calculatorului si mesajele interfetei pot intarzia Tick. In lucrare descriu mecanismele existente, fara a atribui rezultate de performanta nemasurate.

1.6 Tabel comparativ

Tip	Cerinta	Realizare in proiect
Functionala	Deplasare si tintire	GameInputController citeste tastele; Player actualizeaza pozitia si unghiul.
Functionala	Lupta si inamici	CombatSystem gestioneaza proiectilele; EnemyManager actualizeaza inamicii.
Functionala	Progresie	XP, niveluri si o alegere de upgrade pentru fiecare nivel castigat.
Functionala	Navigare in meniuri	ScreenController afiseaza panoul corespunzator starii curente.
Non-functionala	Claritatea interfetei	Text alb pe panouri inchise; bare distincte: rosu pentru HP si cyan pentru XP.
Non-functionala	Stabilitate la resetare	O runda noua goleste listele de obiecte si reseteaza progresul.
Non-functionala	Utilizarea resurselor	Resursele bitmap sunt reutilizate; renderer-ul elibereaza obiectele proprii.

Tip	Cerinta	Realizare in proiect
Non-functionala	Adaptarea ferestrei	Anchor, Dock si recentrarea panourilor gestioneaza redimensionarea.

2. Prezentarea aplicatiei

2.1 Descrierea jocului

Zombie Survivors este un joc 2D de supravietuire realizat in C# Windows Forms. Scopul unei runde este sa rezist cat mai mult, sa elimin inamicii si sa imbunatatesc personajul. Dificultatea creste prin numarul inamicilor din valuri si prin introducerea unor tipuri mai rezistente.

2.2 Functionalitati principale

WASD controleaza deplasarea, **mouse-ul** stabileste tinta, iar **clicul stang** incearca sa traga un proiectil. **ESC** deschide sau inchide pauza, iar **F3** afiseaza Debug Menu. Jocul include HP, XP, Level Up, upgrade-uri, bandaje, Settings, Game Over si Respawn.

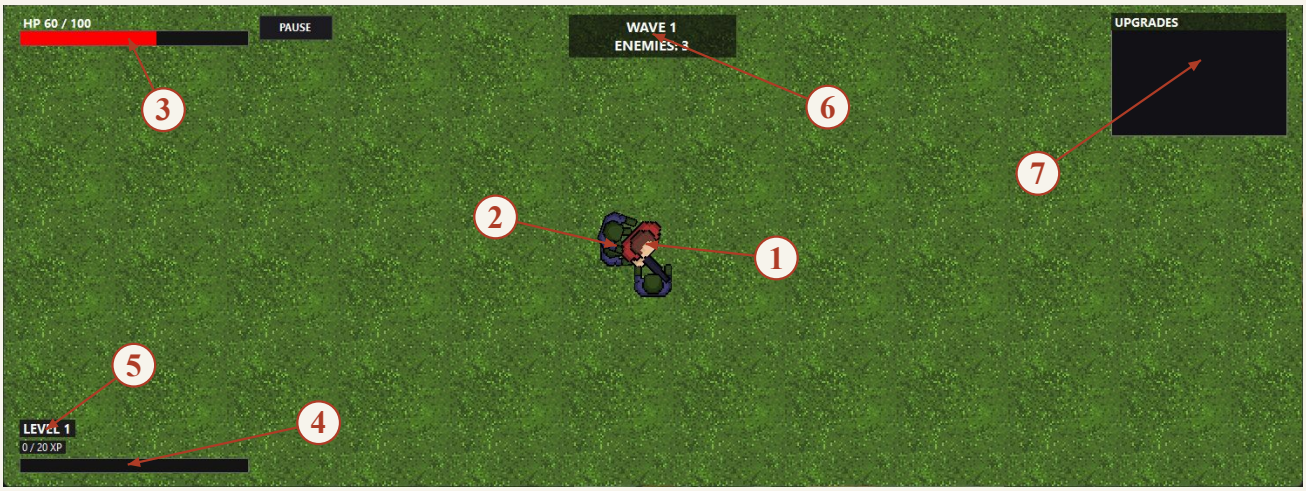
2.3 Tehnologii utilizate

Visual Studio ofera editorul de cod, Designer-ul si instrumentele de proiect. C# exprima regulile jocului, .NET si Windows Forms furnizeaza fereastra si controalele, iar Git si GitHub pastreaza istoricul si repository-ul proiectului.

2.4 Captura generala a jocului

Sagetile indica: 1 - jucatorul; 2 - inamicii; 3 - bara HP; 4 - bara XP; 5 - nivelul; 6 - valul si inamicii ramasi; 7 - lista de upgrade-uri.

Interfata principala a jocului Zombie Survivors; captura reala dintr-o runda.



Solutia, namespace-urile si titlul ferestrei mai contin denumiri vechi. Capturile si fragmentele sunt autentice, de aceea le pastrez. Numele utilizat in aceasta lucrare este Zombie Survivors.

3. Structura proiectului

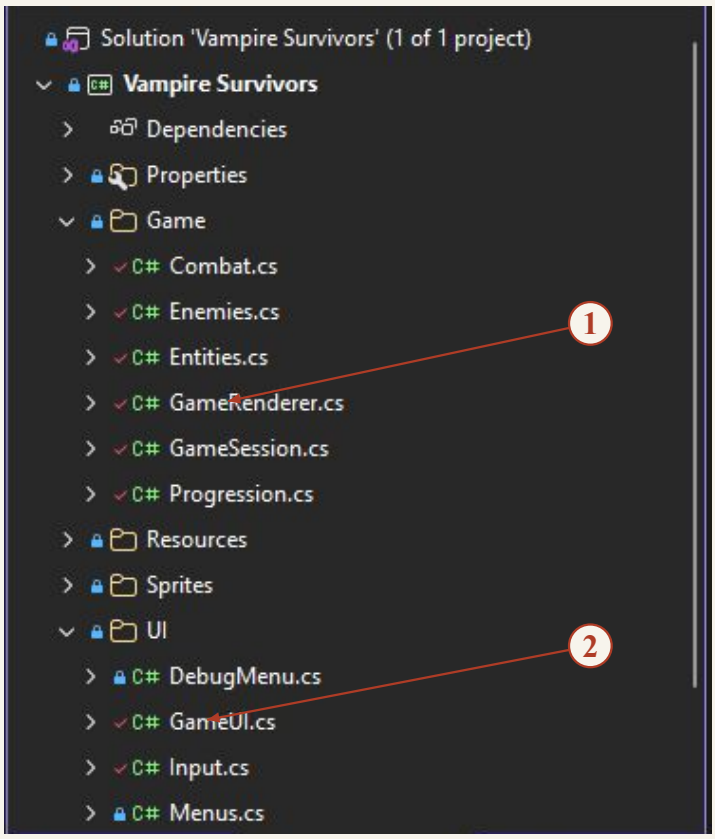
3.1 Organizarea fisierelor

Game/ contine Entities.cs (Player, statistici, proiectile si efecte), Enemies.cs (inamici si valuri), Combat.cs, GameSession.cs, GameRenderer.cs si Progression.cs.

UI/ contine GameUI.cs (HUD si Level Up), Menus.cs (ecrane si afisare), Input.cs (tastatura si mouse) si DebugMenu.cs.

Form1.cs conecteaza aceste clase. **Form1.Designer.cs** defineste controalele. **Program.cs** porneste aplicatia. **Properties/Resources.resx** si directoarele cu imagini furnizeaza resursele grafice.

Fisierele actuale din Game si UI, in Solution Explorer.



3.2 Rolul formularului Form1

Form1 este fereastra principala si punctul de intalnire al componentelor. In constructor creez sesiunea, renderer-ul si controlerele pentru HUD, meniuri, intrare si Debug. Formularul primeste evenimentele Windows Forms si le transmite obiectului responsabil, fara sa pastreze toate regulile de lupta in acelasi fisier.

Clasa este **partial**: Form1.cs si Form1.Designer.cs reprezinta doua parti ale aceleiasi clase. Aceasta separare permite Designer-ului sa gestioneze configuratia controalelor, iar codului scris manual sa organizeze comportamentul lor.

3.3 Legatura dintre componente

Interfata / intrare → Eveniment C# → GameSession → HUD + randare

Un Click sau o tasta produce un eveniment. Controlerul verifica starea ecranului si apeleaza logica sesiunii. Ulterior, HUD-ul citeste valorile noi, iar Invalidate() solicita redesenarea prin OnPaint. In timpul jocului, Timer-ul repeta ciclul de actualizare.

4. Realizarea practica a interfetei

4.1 Crearea proiectului Windows Forms

Pentru a realiza un proiect de acest tip in Visual Studio, deschid **Create a new project**, aleg sablonul **Windows Forms App** pentru limbajul C#, stabilesc numele si folderul solutiei, apoi aleg versiunea .NET. Proiectul analizat foloseste .NET 10 pentru Windows. Pasii descriu modul de creare; capturile din lucrare prezinta proiectul existent. [4]

Dupa creare, deschid Form1 in modul **Designer**. Suprafata formularului imi permite sa asez controale din Toolbox. Pentru configurarea unui control folosesc fereastra Properties. Pentru comportament trec in Form1.cs si scriu metodele asociate evenimentelor.

Fisier	**Rol la pornire si in interfata**
Program.cs	Main() initializeaza configuratia aplicatiei si executa Application.Run(new Form1()).
Form1.Designer.cs	Declara campurile controalelor si implementeaza InitializeComponent(), cu obiecte, proprietati, ierarhie si evenimente.
Form1.cs	Apeleaza InitializeComponent(), construieste controlerele si conecteaza logica jocului la formular.

Codul 1. Initializarea formularului

Form1.cs:20-24
Form1() • inceputul constructorului; continuarea este omisa

```
public Form1()
{
    InitializeComponent();

    DoubleBuffered = true;
```

InitializeComponent() trebuie executata inainte ca programul sa foloseasca lblHealth, btnPlay sau celelalte controale. Ea creeaza obiectele si le adauga in containerele potrivite. Daca un eveniment din Designer indica o metoda care nu mai exista, proiectul sau Designer-ul pot raporta o eroare.

DoubleBuffered activeaza desenarea intermediara intr-un buffer. In acest joc ajuta la reducerea palpairii in timpul redesenarii arenei. Proprietatea nu inlocuieste actualizarea logicii si nu fixeaza numarul de cadre pe secunda.

De la formular gol la interfata

Am organizat interfata in panouri: meniul principal, setarile, pauza, Level Up si Game Over. Pe formular raman si controalele HUD-ului. Panourile reprezinta ecrane ale aceleiasi ferestre; navigarea schimba vizibilitatea lor, fara sa deschida un formular nou pentru fiecare meniu.

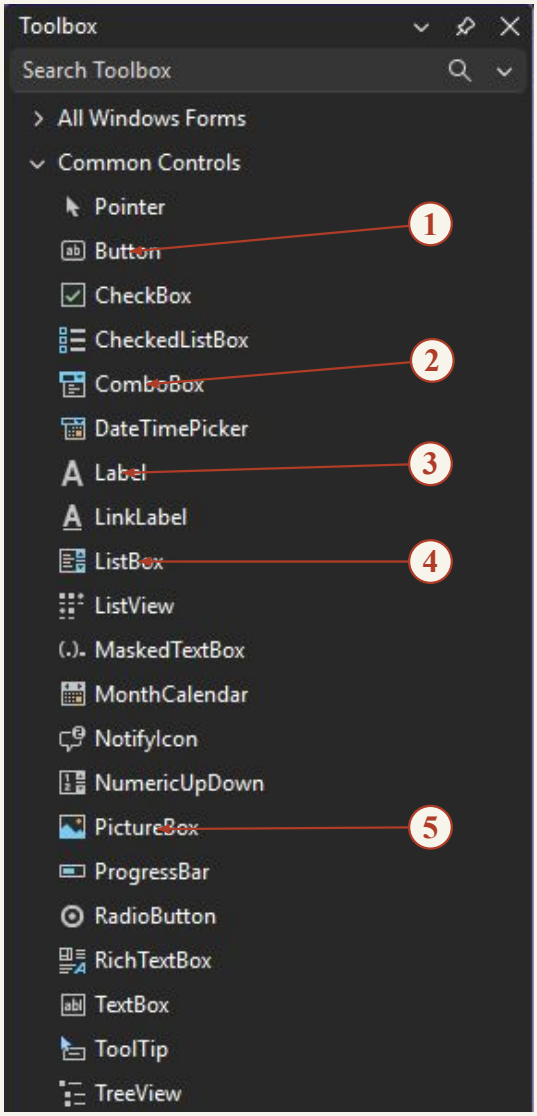
Mai intai stabilesc containerele Panel, apoi adaug textele si butoanele, configurez proprietatile si, la final, conectez evenimentele. Ierarhia parintilor este importanta: Location este relativa la containerul controlului.

4.2 Utilizarea Toolbox-ului

Toolbox este lista de controale si componente disponibile pentru Designer. Il pot deschide din View → Toolbox sau prin Ctrl+Alt+X. Aleg un element si il plasez pe formular ori in interiorul unui Panel. Controlul selectat se configureaza apoi in Properties. [2, 4]

Control	**Utilizare efectiva**
Button	PLAY, SETTINGS, RESUME, RESPAWN si cele trei upgrade-uri.
Label	HP, XP, nivel, val, titluri si informatii Debug.
Panel	Ecrane, grupuri de controale si cele doua straturi ale barelor.
PictureBox	Logo-ul meniului: Image din resurse, SizeMode = Zoom.
CheckBox	Infinite Health si Infinite XP in Debug.
ComboBox	Lista inchisa a modurilor de afisare.
ListBox	Upgrade-urile detinute, in partea dreapta-sus.
Timer	Declanseaza periodic evenimentul Tick al jocului.

Toolbox deschis in Visual Studio: 1 - Button; 2 - ComboBox; 3 - Label; 4 - ListBox; 5 - PictureBox.



De ce am ales aceste controale

Button ofera direct evenimentul Click. Label este potrivit pentru informatii scurte, iar ListBox poate afisa un numar variabil de upgrade-uri. Panel grupeaza controalele si permite ascunderea intregului meniu. ComboBox cu DropDownList impiedica introducerea unei valori care nu exista in lista.

Pentru fiecare control stabilesc un **Name** clar, apoi Text, Size, Location, culori si font. La butoane configurez si aspectul la trecerea mouse-ului. Pentru controalele HUD aleg Anchor in functie de marginea ferestrei langa care trebuie sa ramana.

Este o componenta Windows Forms, dar gameTimer este construit in Form1.cs, cu Interval = 16 si Tick += GameTimer_Tick. Nu este un control vizibil si nu a fost adaugat in fisierul Designer.

4.3 Form Designer si Properties

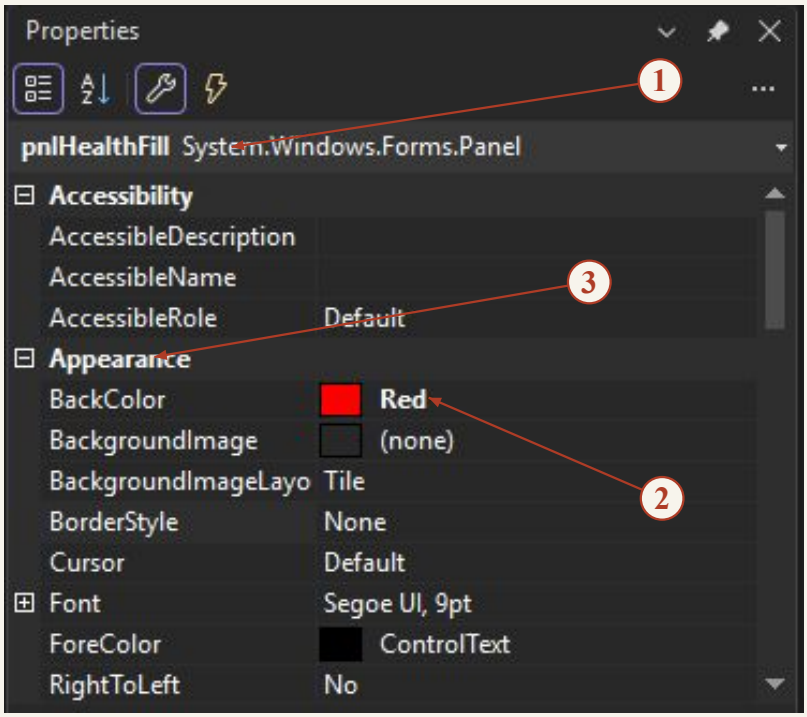
In Designer selectez un control pentru a-i vedea conturul si proprietatile. Properties modifica configuratia obiectului selectat; Document Outline arata ierarhia, utila cand mai multe panouri se suprapun. In captura de mai jos este selectat pnlHealthFill, de tip Panel.

Name este numele folosit in cod, de exemplu pnlHealthFill. **Text** este continutul afisat, ca PLAY sau HP 100 / 100. Un Panel folosit ca bara nu are nevoie de text.

Size inseamna latime si inaltime. **Location** este pozitia in interiorul parintelui. pnlHealthFill incepe la (0, 0) in panoul de fundal; nu foloseste coordonatele intregului ecran.

BackColor coloreaza fundalul, **ForeColor** textul, iar **Font** stabileste familia, marimea si stilul literelor.

Properties pentru pnlHealthFill: 1 - obiectul selectat; 2 - BackColor = Red; 3 - categoria Appearance.



Proprietate	**Exemplu si efect in Zombie Survivors**
Anchor	Bottom | Left pentru lblLevel, lblXp si pnlXpBackground; pastreaza distanta fata de stanga si de jos.
Dock	Fill pentru pnlMainMenu si pnlSettings; panoul ocupa suprafata disponibila a formularului.
Visible	pnlLevelUp porneste ascuns; ScreenController il afiseaza la alegerea unui upgrade.
Font / culori	btnPlay foloseste Segoe UI 12, Bold, text alb si fundal inchis.
Size / Location	picLogo are 350 × 150 si pozitia (44, 22) in pnlMenuContent, in configuratia Designer.

Click este un eveniment

Click nu este o proprietate obisnuita, precum BackColor. Este un eveniment. In Properties trec la lista de evenimente, prin pictograma fulger, si asociez o metoda. Legatura reala a butonului PLAY este **btnPlay.Click += BtnPlay_Click;**. La apasare, Windows Forms executa metoda respectiva. [5]

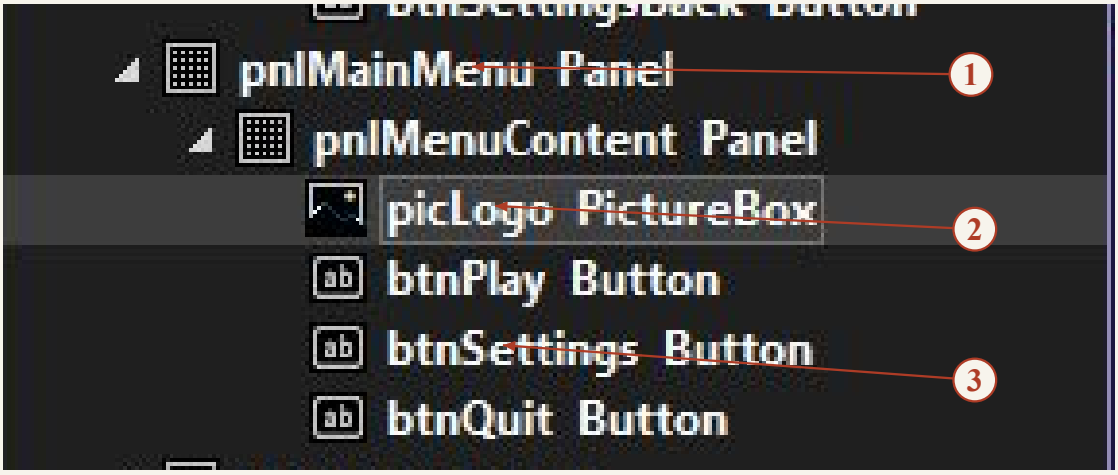
Controalele ascunse in timpul rularii pot fi totusi vizibile sau suprapuse in Designer. Aspectul de proiectare nu trebuie confundat cu starea curenta a jocului.

4.4 Crearea meniului principal

Am grupat meniul in **pnlMainMenu**, cu Dock = Fill. In interior se afla **pnlMenuContent**, care contine picLogo, btnPlay, btnSettings si btnQuit. ScreenController recentreaza grupul dupa dimensiunea ferestrei; pozitiile butoanelor raman relative la acest grup.

In Designer, logo-ul selectat apartine meniului chiar daca alt panou il acopera pe suprafata de proiectare.

Meniul in Document Outline al Designer-ului: 1 - container; 2 - PictureBox; 3 - butoane.



Meniul la rulare: logo, PLAY, SETTINGS si QUIT.



picLogo incarca Properties.Resources.Logo. Modul Zoom pastreaza proportiile imaginii in spatiul disponibil. Logo-ul este static in versiunea actuala; nu exista o animatie de meniu care trebuie descrisa.

Codul 2. Actiunea butonului PLAY

Form1.cs:104-104; Form1.cs:147-151
BtnPlay_Click() si StartNewGame()

```
private void BtnPlay_Click(object? sender, EventArgs e) => StartNewGame();
// ... portiune intermediara omisa ...
private void StartNewGame()
{
    RestartGame();
    gameTimer.Start();
}
```

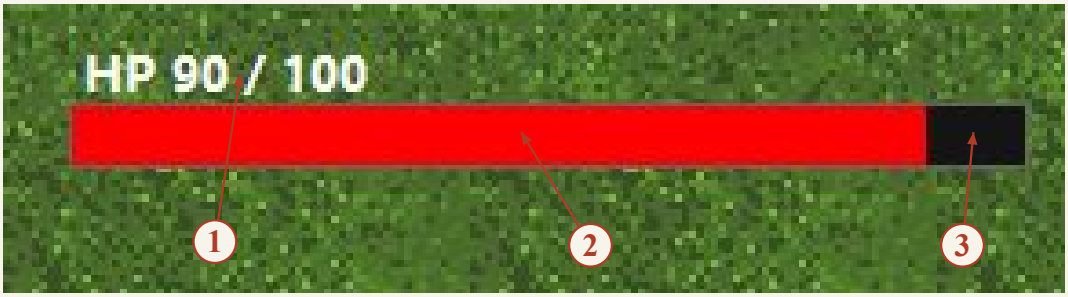
Evenimentul Click ajunge in **BtnPlay_Click**, care apeleaza StartNewGame(). Mai intai RestartGame() pregateste sesiunea, apoi gameTimer.Start() porneste actualizarile. SETTINGS afiseaza panoul de setari, iar QUIT executa Application.Exit().

Butoanele folosesc FlatStyle = Flat, Cursor = Hand, text alb si culori distincte la hover sau apasare. Aceste proprietati ofera feedback fara sa necesite desenarea manuala a fiecarui buton.

4.5 Crearea barei HP

Pentru bara de viata am folosit doua controale Panel. **pnlHealthBackground** este fundalul intunecat, cu bordura, iar **pnlHealthFill** este partea rosie din interior. Deasupra se afla **lblHealth**, care afiseaza viata curenta si limita maxima.

Bara HP dupa o lovitura: 1 - textul; 2 - umplerea rosie; 3 - fundalul ramas liber.



In Designer, fundalul are Size = 263 × 18, iar umplerea incepe la Location = (0, 0). Culoarea umplerii este Red. Latimea initiala din Designer este numai o valoare de proiectare: la rulare, metoda UpdateBar() o calculeaza din viata jucatorului.

Codul 3. Actualizarea vietii si a latimii barei

UI/GameUI.cs:41-48; UI/GameUI.cs:65-71
HudController.UpdateHealth() si UpdateBar()

```
public bool UpdateHealth()
{
    Player player = session.Player;
    UpdateBar(healthFill, player.Health, player.MaxHealth);
    healthLabel.Text = $"HP {player.Health} / {player.MaxHealth}";
    return player.IsDead;
}

// ... portiune intermediara omisa ...
private static void UpdateBar(Panel fill, int value, int maximum)
{
    // Resize only the fill panel; the labels stay above the bar.
    float fraction = maximum > 0 ? value / (float)maximum : 0f;
    fraction = Math.Clamp(fraction, 0f, 1f);
    fill.Width = (int)(fill.Parent!.ClientSize.Width * fraction);
}
```

player.Health reprezinta HP curent, iar **player.MaxHealth** limita data de statistici. Formula este **procent HP = HP curent / HP maxim**. Conversia la float permite un rezultat fractionar. Math.Clamp limiteaza rezultatul intre 0 si 1, iar verificarea maximum > 0 evita impartirea la zero.

Procentul inmulteste latimea interioara a panoului parinte. De exemplu, pentru 90 / 100 HP si o zona interioara de 261 px, latimea calculata este 234 px dupa conversia la int. Se schimba numai umplerea; label-ul si fundalul isi pastreaza pozitia.

UpdateHealth() returneaza si player.IsDead. GameTimer_Tick foloseste rezultatul primit prin RefreshFrame() pentru a afisa panoul Game Over cand viata ajunge la zero.

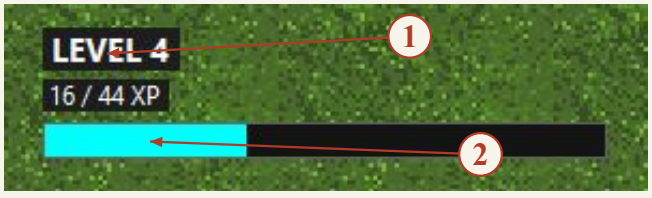
4.6 Bara XP si nivelul

In partea stanga-jos am plasat **lblLevel**, **lblXp** si **pnlXpBackground**. In panoul de fundal se afla pnlXpFill, de culoare Cyan. Cele trei controale exterioare au Anchor = Bottom | Left; astfel isi pastreaza distantele fata de marginile de jos si din stanga la redimensionare.

La pornire: nivel 1; sageata 1 indica 0 / 20 XP.



Dupa +100 XP: 1 - nivelul 4; 2 - bara 16 / 44 XP.



HudController.UpdateExperience() citeste Player, actualizeaza textele si apeleaza aceeasi metoda UpdateBar() ca la HP. Aici fractia este **Experience / ExperienceToNextLevel**. Dupa eliminarea unui inamic, CombatSystem acorda XP, iar sesiunea inregistreaza nivelurile nou castigate.

Codul 4. Calculul progresului in Player

Game/Entities.cs:158-176
Player.AddExperience()

```
public int AddExperience(int amount)
{
    if (amount <= 0 || IsDead)
        return 0;

    Experience += amount;
    int levelsGained = 0;

    // Keep leftover XP, even if one reward gains several levels.
    while (Experience >= ExperienceToNextLevel)
    {
        Experience -= ExperienceToNextLevel;
        Level++;
        ExperienceToNextLevel += 8;
        levelsGained++;
    }

    return levelsGained;
}
```

amount este recompensa primita. **Experience** pastreaza XP acumulat, iar **ExperienceToNextLevel** incepe de la 20 si creste cu 8 dupa fiecare nivel. Bucla while pastreaza surplusul: o recompensa mare poate acorda mai multe niveluri, fara sa piarda XP ramas.

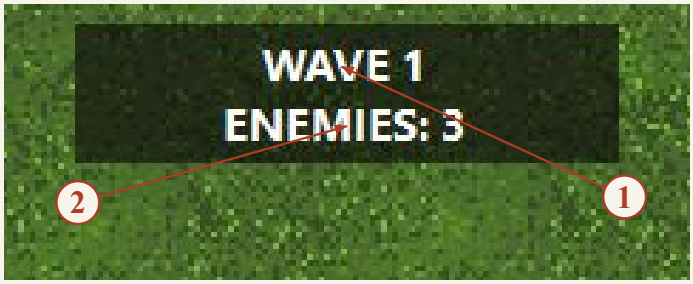
In captura reala, comanda Debug +100 XP a trecut prin pragurile 20, 28 si 36: 100 − 20 − 28 − 36 = 16. De aceea HUD-ul arata LEVEL 4 si 16 / 44 XP. Meniurile de alegere pentru nivelurile 2, 3 si 4 sunt prezentate pe rand, prin coada PendingLevels.

Player.Reset() readuce Level la 1, Experience la 0 si pragul la 20. RestartGame() cere imediat actualizarea HUD-ului, pentru ca valorile vechi sa nu ramana pe ecran.

4.7 Interfata pentru valuri

Pentru valuri folosesc un singur **Label**, lblWave, cu doua randuri de text si aliniere centrala. Primul rand indica numarul valului, iar al doilea numarul inamicilor ramasi. In pauza dintre valuri, acelasi label anunta finalizarea si timpul pana la urmatorul val.

Detaliu al lblWave: 1 - valul curent; 2 - totalul inamicilor ramasi in val.



Codul 5. Actualizarea informatiilor despre val

UI/GameUI.cs:73-81
HudController.UpdateWave()

```
public void UpdateWave()
{
    WaveManager waves = session.Waves;
    int seconds = (waves.IntermissionRemainingMs + 999) / 1000;
    waveLabel.Text = waves.IsWaveActive
        ? $"WAVE {waves.CurrentWave}{Environment.NewLine}ENEMIES:
    {waves.GetEnemiesRemaining(session.Enemies.Count)}"
        : $"WAVE {waves.CurrentWave} CLEARED{Environment.NewLine}NEXT WAVE:
    {seconds}s";
}
```

session.Waves este managerul valurilor. **CurrentWave** este numarul valului, iar **IsWaveActive** selecteaza textul afisat. Environment.NewLine separa randurile. Pentru pauza, expresia (milisecunde + 999) / 1000 rotunjeste in sus numarul de secunde, astfel incat un rest de timp sa nu fie afisat ca zero.

Element	Semnificatie pentru interfata
WAVE 1	Valul din care fac parte inamicii si aparitiile inca programate.
ENEMIES: 3	Include inamicii vii si pe cei din val care inca nu au aparut.
WAVE … CLEARED	Valul s-a incheiat si se executa pauza dintre valuri.
NEXT WAVE: …s	Timpul ramas pana la inceperea urmatorului val.

Pozitionarea la redimensionare

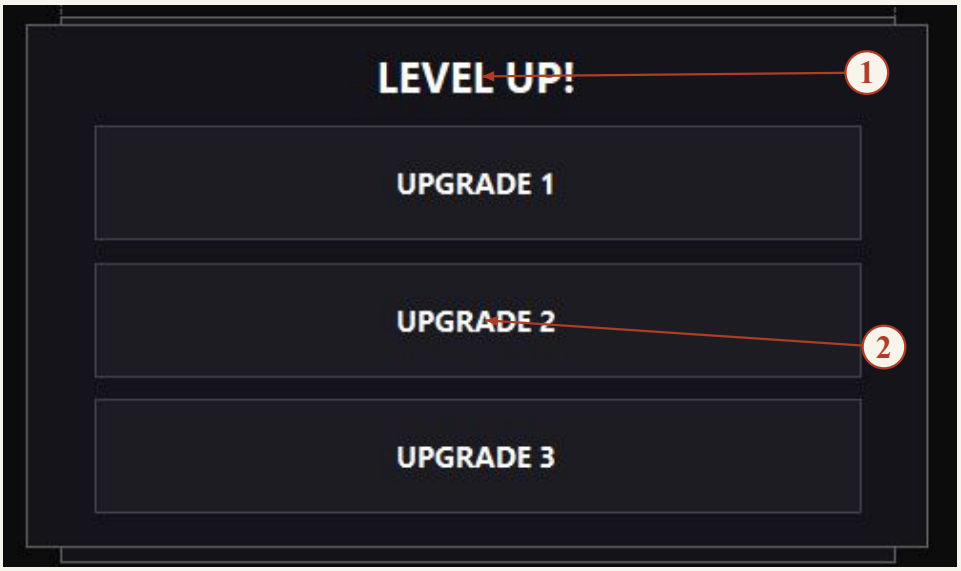
HudController.Layout() centreaza label-ul pe orizontala. Daca spatiul este prea mic langa butonul Pause sau lista de upgrade-uri, muta textul mai jos. Astfel, pozitia este stabilita dupa dimensiunea curenta a ferestrei, nu doar dupa coordonatele initiale din Designer.

Label-ul nu creeaza inamici si nu avanseaza valul. El citeste datele din WaveManager. Aceasta separare face mai usoara modificarea textului fara schimbarea regulilor jocului.

4.8 Interfata Level Up

Panoul **pnlLevelUp** contine lblLevelUpTitle si trei Button: btnUpgrade1, btnUpgrade2, btnUpgrade3. Are fundal inchis, bordura si Visible = false in configuratia initiala. ScreenController il centreaza si il aduce in fata cand starea devine LevelUp.

Panoul Level Up in Designer: titlu si trei butoane.



Optiuni generate la rulare pentru nivelul 2.



Codul 6. Completarea textelor celor trei butoane

UI/GameUI.cs:150-162
LevelUpController.ShowNext() • fragmentul care pregateste optiunile

```
int level = session.PendingLevels.Peek();
// Every level offers three different normal upgrades.
upgradeChoices = session.Upgrades.GetRandomUpgrades(buttons.Length);
title.Text = $"LEVEL {level} — CHOOSE AN UPGRADE";
for (int index = 0; index < buttons.Length; index++)
{
    Button button = buttons[index];
    button.Visible = true;
    UpgradeType choice = upgradeChoices[index];
    (string name, string description) = UpgradeSystem.GetUpgradeInfo(choice);
    button.Text = $"{name}{Environment.NewLine}{description}   Owned:
    {session.Upgrades.GetStackCount(choice)}";
}
screens.ShowLevelUp();
```

PendingLevels.Peek() indica nivelul pentru care trebuie facuta alegerea. **buttons.Length** este 3. GetRandomUpgrades() alege din lista upgrade-urilor normale si scoate optiunea aleasa din grupul temporar, astfel incat acelasi meniu sa nu contina duplicate.

In bucla, fiecare buton devine vizibil si primeste numele upgrade-ului, descrierea si numarul de exemplare detinute, afisat prin **Owned**. Textele initiale UPGRADE 1-3 sunt doar valori de proiectare; la rulare sunt inlocuite cu informatiile optiunilor reale.

Cat timp starea este LevelUp, GameTimer_Tick se opreste inaintea actualizarii sesiunii. Jucatorul poate citi si alege fara ca inamicii sa continue sa-l atace.

Aplicarea si afisarea upgrade-urilor

Fiecare buton are propriul eveniment Click. Metodele BtnUpgrade1_Click, BtnUpgrade2_Click si BtnUpgrade3_Click transmit indicii 0, 1 si 2 catre ChooseLevelUpOption(). Alegerea ajunge apoi in LevelUpController.Choose(), care valideaza starea inainte sa modifice jucatorul.

Codul 7. Aplicarea alegerii si continuarea meniurilor

UI/GameUI.cs:165-179
LevelUpController.Choose()

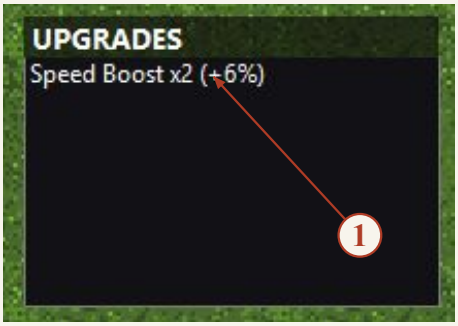
```
public bool Choose(int index, bool infiniteXp)
{
    if (screens.State != GameState.LevelUp || session.PendingLevels.Count == 0 ||
        index < 0 || index >= upgradeChoices.Count)
        return false;
    session.Upgrades.ApplyUpgrade(upgradeChoices[index], session.Player);
    session.PendingLevels.Dequeue();
    if (infiniteXp)
        session.GrantNextLevel();
    hud.UpdateHealth();
    hud.UpdateExperience();
    hud.UpdateUpgrades();
    ShowNext();
    return true;
}
```

index este pozitia butonului ales, iar **upgradeChoices** contine optiunile curente. ApplyUpgrade() modifica statisticile si numarul de stive; Dequeue() elimina nivelul rezolvat. La final sunt actualizate HP, XP si lista de bonusuri, apoi ShowNext() verifica daca mai exista o alegere in asteptare.

lstUpgrades este un ListBox, ancorat sus-dreapta. Fiecare tip de upgrade apare pe un singur rand, cu numarul de stive si bonusul total.

HudController.UpdateUpgrades() citeste Version. Daca versiunea nu s-a schimbat, lista nu este reconstruita inutil la fiecare Tick.

Lista din dreapta-sus dupa doua alegeri Speed Boost: x2, total +6%.



Cand dispare panoul?

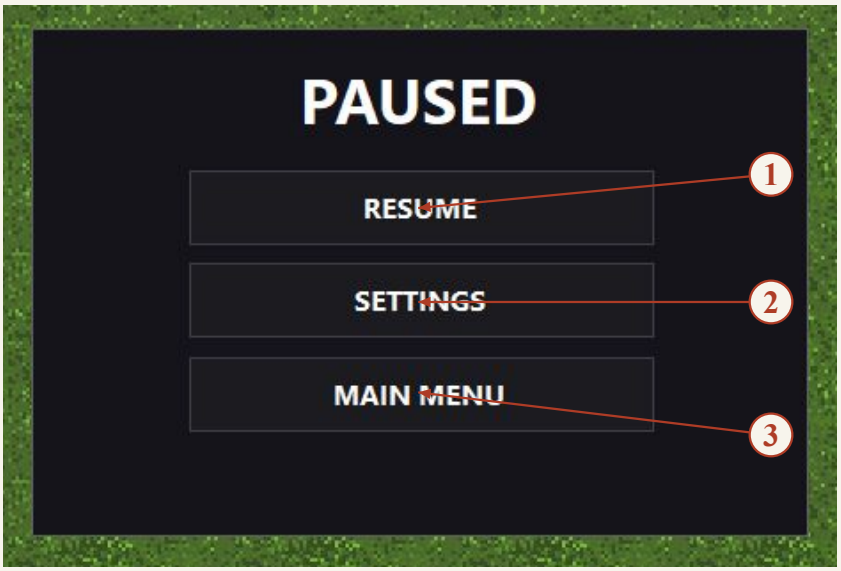
Daca mai exista niveluri in coada, ShowNext() pregateste un nou set de trei optiuni. Daca nu mai exista, CloseLevelUp() trece la Playing si ascunde panoul. Cand jucatorul este mort, aceeasi verificare duce la GameOver. In modul Debug, infiniteXp poate acorda inca un nivel dupa fiecare alegere.

Speed Boost adauga de fiecare data 3% la bonusul de viteza. Doua alegeri dau +6%, nu 1,03 × 1,03. La viteza de baza 6, rezultatul este 6 × 1,06 = 6,36 unitati pe actualizare.

4.9 Pause Menu

Meniul de pauza este **pnlPause**, cu lblPaused si butoanele RESUME, SETTINGS si MAIN MENU. Este centrat de ScreenController. Acelasi meniu se poate deschide din butonul PAUSE al HUD-ului sau prin ESC in timpul jocului.

Meniul Pause: 1 - RESUME; 2 - SETTINGS; 3 - MAIN MENU.



Codul 8. Schimbarea starii pentru pauza si reluare

UI/Menus.cs:63-78
ScreenController.Pause() si Resume()

```
public bool Pause()
{
    if (State != GameState.Playing)
        return false;
    SetState(GameState.Paused);
    return true;
}

public bool Resume()
{
    if (State != GameState.Paused)
        return false;
    SetState(GameState.Playing);
    return true;
}
```

Pause() accepta cererea doar din Playing, iar Resume() doar din Paused. In Form1, PauseGame() si ResumeGame() sterg tastele de miscare retinute. Astfel, reluarea nu porneste o deplasare ramasa dinaintea pauzei. La reluare se verifica si existenta nivelurilor in asteptare.

Pauza nu inseamna oprirea si recrearea Timer-ului. In GameTimer_Tick, prima verificare returneaza imediat daca starea nu este Playing. Datele rundei raman in memorie, dar miscarea, lupta

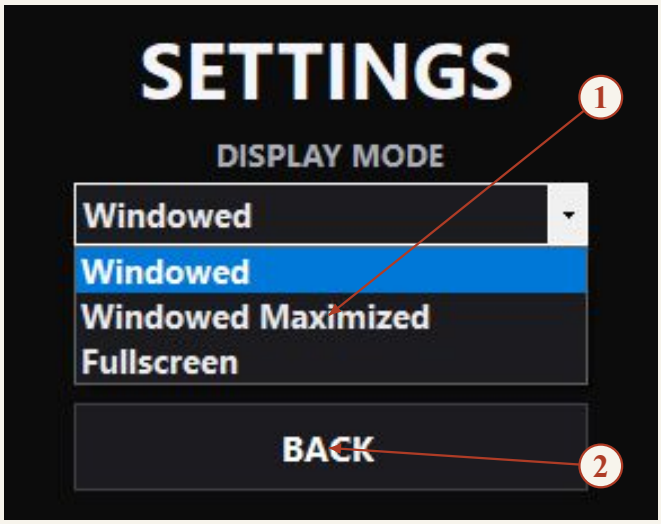
si valurile nu mai sunt actualizate. KeyDown gestioneaza ESC o singura data pentru fiecare apasare, folosind setul de taste apasate.

SETTINGS retine ca revenirea trebuie facuta la pauza. MAIN MENU opreste Timer-ul, reinitializeaza sesiunea si afiseaza meniul principal. RESUME continua runda curenta.

4.10 Settings

Setarile sunt grupate in **pnlSettings** si pnlSettingsContent. Panoul interior contine titlul, eticheta DISPLAY MODE, **cmbDisplayMode** si butoanele APPLY si BACK. ComboBox are DropDownStyle = DropDownList, deci utilizatorul poate alege numai una dintre cele trei valori existente.

Setarile reale: 1 - lista celor trei moduri; 2 - butonul BACK.



Mod	Efect implementat de DisplayManager
Windowed	Formular cu bordura redimensionabila; pozitia si dimensiunea ferestrei sunt memorate pe durata sesiunii aplicatiei.
Windowed Maximized	Formular cu bordura redimensionabila si WindowState = Maximized.
Fullscreen	Formular fara bordura, asezat pe dreptunghiul complet al monitorului curent.

Aplicarea unei alegeri

Click pe APPLY ajunge in BtnApplySettings_Click(), apoi in display.ApplySelectedMode(). SelectedIndex este convertit la DisplayMode, iar Apply() modifica FormBorderStyle, WindowState si Bounds. Pentru Windowed, dimensiunea si pozitia sunt limitate la zona de lucru disponibila a monitorului.

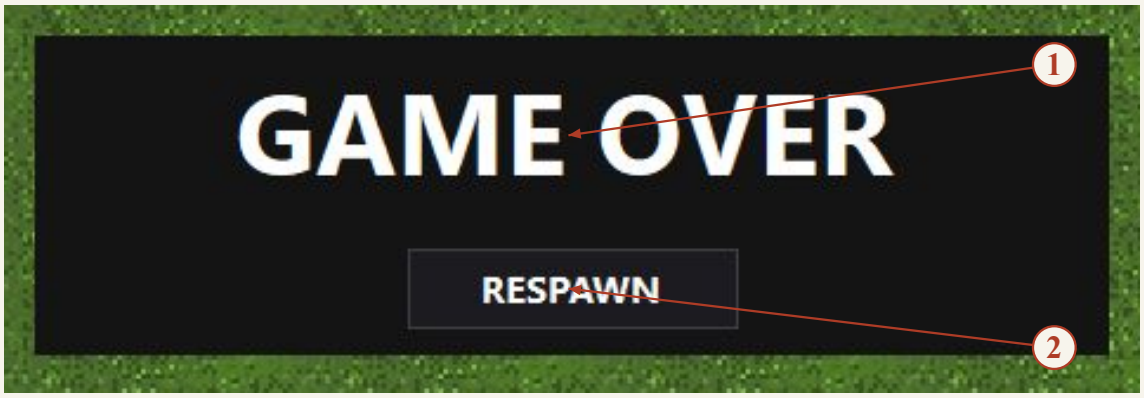
Variabila **applying** impiedica tratarea schimbarilor intermediare de dimensiune ca modificari facute manual. La redimensionare, Form1 cere recentrarea panourilor si actualizarea dimensiunii arenei. Nu exista in acest ecran setari de volum sau de dificultate.

Din meniul principal, BACK revine la MainMenu. Daca Settings a fost deschis din Pause, BACK revine la Paused. Aceasta alegere este pastrata in SettingsReturnToPause; nu se pierde runda aflata in pauza.

4.11 Game Over si Respawn

Cand HP ajunge la zero, Player.IsDead devine true. Actualizarea HUD-ului raporteaza moartea, iar Form1 apeleaza screens.ShowGameOver(). **pnlGameOver** afiseaza titlul si butonul RESPAWN. Starea GameOver opreste actualizarile de joc prin aceeasi verificare de la inceputul Tick-ului.

Game Over real: 1 - mesajul; 2 - butonul RESPAWN.



Codul 9. Reinitializarea sesiunii si a HUD-ului

Form1.cs:162-176
Form1.RestartGame()

```
private void RestartGame()
{
    // The form may finish maximizing after the session was created.
    session.Resize(ClientSize);
    session.Reset();
    gameInput.Reset();
    screens.SetPlaying();
    hud.UpdateHealth();
    hud.UpdateExperience();
    hud.UpdateUpgrades();
    hud.UpdateWave();
    debug.UpdateInfo();
    Invalidate();
}
```

session.Resize(ClientSize) foloseste dimensiunea actuala a ferestrei. Reset() porneste o sesiune curata, iar gameInput.Reset() elimina tastele retinute. SetPlaying() ascunde Game Over si readuce starea de joc. Cele patru apeluri HUD afiseaza imediat viata, XP, upgrade-urile si valul resetat.

Date resetate	Rezultatul unei runde noi
Jucator si progres	HP 100 / 100, nivel 1, 0 / 20 XP, statistici de baza si fara upgrade-uri.

Date resetate	Rezultatul unei runde noi
Lumea rundei	Fara inamici vechi, bandaje, proiectile, texte de lupta sau particule.
Valuri si alegeri	Valul 1, contorul de aparitii resetat, PendingLevels golit.

BtnRespawn_Click() apeleaza RestartGame(). Timer-ul ramane pornit in GameOver, iar revenirea la Playing permite actualizarea. Fluxul complet este explicat in subcapitolul 4.13.

4.12 Debug Menu

Meniul Debug se deschide prin **F3** si este un Panel cu etichete, doua CheckBox-uri si cinci butoane. DebugController.Toggle() il ascunde sau il afiseaza si il aduce in fata. Nu il deschide din MainMenu sau Settings.

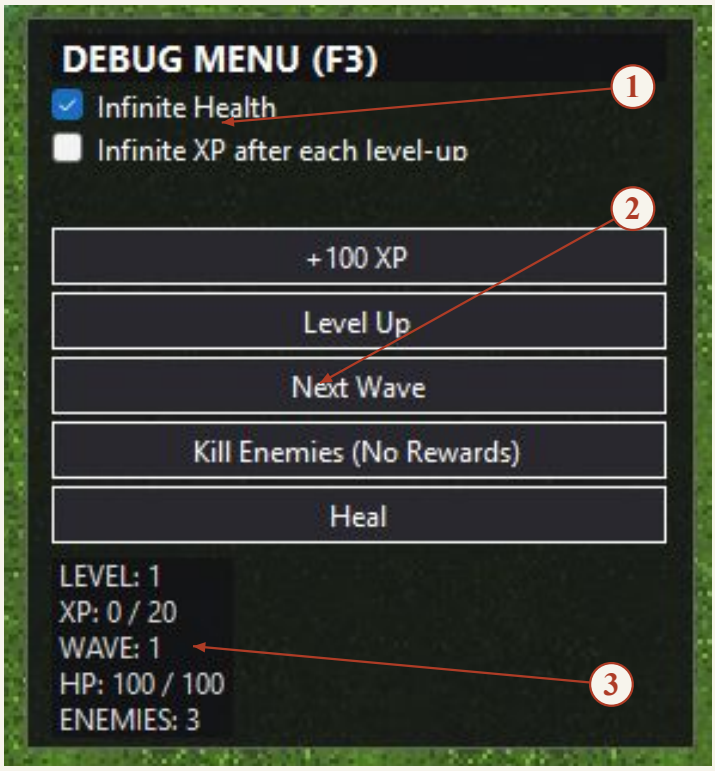
Infinite Health previne damage-ul de contact si reface viata daca personajul este inca viu.

Infinite XP acorda inca un nivel dupa fiecare alegere de upgrade. Nu este o valoare numerica infinita in proprietatea XP.

+100 XP / Level Up acorda experienta sau exact cat lipseste pentru pragul urmator.

Next Wave curata inamicii si avanseaza valul. **Kill Enemies (No Rewards)** incheie valul fara recompense. **Heal** vindeca un personaj viu.

Meniul Debug real: 1 - CheckBox-uri; 2 - instrumente; 3 - datele sesiunii.



Codul 10. Acordarea de XP prin interfata Debug

UI/DebugMenu.cs:55-65
DebugController.AddExperience()

```
public void AddExperience(int amount)
{
    if (amount <= 0 || session.Player.IsDead)
        return;
    session.AddExperience(amount);
    hud.UpdateExperience();
    if (session.PendingLevels.Count > 0 && screens.State == GameState.Playing)
        levels.ShowNext();
    UpdateInfo();
    repaint();
}
```

Metoda ignora valorile nepozitive si jucatorul mort. Dupa session.AddExperience(amount), actualizeaza bara XP si deschide Level Up daca exista alegeri si jocul este Playing. Apoi reimprospateaza informatiile Debug si solicita redesenarea prin callback-ul repaint().

Acest meniu permite verificarea rapida a nivelurilor, stivelor si valurilor. Capturile progresiei din lucrare au fost obtinute prin aceste controale existente, fara schimbarea surselor sau simularea unor ecrane.

4.13 Legatura dintre UI si cod

In Windows Forms, un buton nu functioneaza doar pentru ca apare pe formular. El trebuie sa existe ca obiect, sa fie adaugat intr-un container si sa aiba evenimentul conectat. Pentru PLAY pot urmari toate aceste etape in fisierele reale ale proiectului.

Declarare, creare si proprietati

In Form1.Designer.cs, campul este declarat **private System.Windows.Forms.Button btnPlay;**. In InitializeComponent() este creat prin **btnPlay = new Button();** si adaugat in pnlMenuContent. Aceste instructiuni apartin aceleiasi clase partial Form1 folosite si de codul manual.

Codul 11. Proprietatile butonului si legarea evenimentului

Form1.Designer.cs:302-317
InitializeComponent() • configuratia reala a btnPlay

```
btnPlay.BackColor = Color.FromArgb(28, 28, 32);
btnPlay.Cursor = Cursors.Hand;
btnPlay.FlatAppearance.BorderColor = Color.FromArgb(65, 65, 75);
btnPlay.FlatAppearance.MouseDownBackColor = Color.FromArgb(20, 20, 24);
btnPlay.FlatAppearance.MouseOverBackColor = Color.FromArgb(46, 46, 54);
btnPlay.FlatStyle = FlatStyle.Flat;
btnPlay.Font = new Font("Segoe UI", 12F, FontStyle.Bold);
btnPlay.ForeColor = Color.White;
btnPlay.Location = new Point(105, 210);
btnPlay.Margin = new Padding(3, 2, 3, 2);
btnPlay.Name = "btnPlay";
btnPlay.Size = new Size(228, 39);
btnPlay.TabIndex = 0;
btnPlay.Text = "PLAY";
btnPlay.UseVisualStyleBackColor = false;
btnPlay.Click += BtnPlay_Click;
```

Name identifica obiectul in cod, **Text** stabileste inscriptia PLAY, iar Size si Location definesc dreptunghiul din container. Font si ForeColor controleaza textul. Proprietatile FlatAppearance stabilesc bordura si feedback-ul la hover sau apasare.

Apasare PLAY → Click → BtnPlay_Click() → StartNewGame()

Operatorul += aboneaza metoda BtnPlay_Click la eveniment. Parametrul sender reprezinta obiectul care a produs evenimentul, iar EventArgs e transporta datele generale ale acestuia. Metoda din Form1.cs transmite actiunea la StartNewGame(), prezentata in Codul 2.

RestartGame() reseteaza datele, SetPlaying() ascunde meniul si face HUD-ul vizibil, apoi Start() permite actualizarile Timer-ului. Un singur Click se transforma astfel intr-o runda noua, pregatita complet.

De la RESPAWN la o runda noua

Al doilea exemplu este **btnRespawn**. Campul de tip Button este declarat in Form1.Designer.cs, creat in InitializeComponent() si adaugat in pnlGameOver. Text este RESPAWN, iar Name este btnRespawn. Linia **btnRespawn.Click += BtnRespawn_Click;** conecteaza butonul la codul manual.

RESPAWN → Click → BtnRespawn_Click() → RestartGame()

Runda imediat dupa RESPAWN: 1 - HP refacut; 2 - XP si nivel resetate; 3 - valul 1; 4 - lista de upgrade-uri goala.



Ce face fiecare nivel al aplicatiei?

Nivel	Responsabilitate in acest exemplu
Designer	Pastreaza obiectul Button, proprietatile, parintele si asocierea evenimentului.
Form1.cs	Primeste Click prin BtnRespawn_Click si coordoneaza RestartGame().
GameSession	Goleste obiectele rundei, reseteaza jucatorul, valurile si upgrade-urile.
ScreenController / HUD	Afiseaza ecranul de joc si valorile initiale, apoi solicita redesenarea.

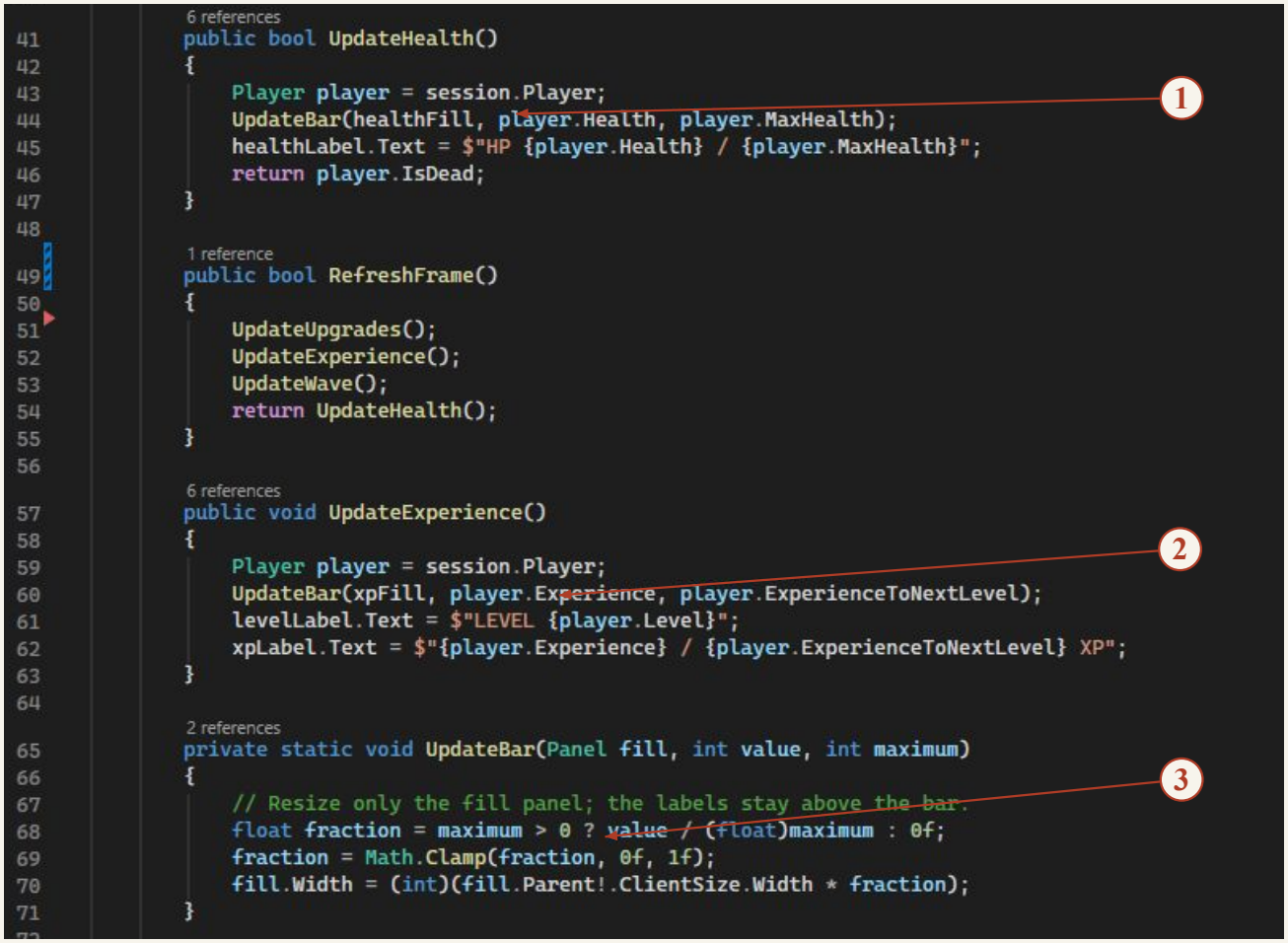
Separarea este utila cand modific interfata: pot schimba inscriptia sau pozitia butonului in Designer fara sa rescriu resetarea sesiunii. Invers, pot completa regulile de resetare in GameSession fara sa mut butonul sau sa modific resursa grafica a meniului.

Si btnUpgrade1 are o asociere Click in Designer. Metoda sa trimite indicele 0, iar LevelUpController valideaza alegerea si modifica statisticile. Controlul vizual doar initiaza actiunea; regulile raman in clasele corespunzatoare.

Integrarea interfetei cu jocul

HudController primeste referinte la controalele create in Designer si la GameSession. Astfel, citeste aceleasi date ca logica jocului si le prezinta prin Text sau Width. Captura editorului arata metodele reale UpdateHealth(), UpdateExperience() si UpdateBar().

Codul HUD-ului in Visual Studio: 1 - viata; 2 - XP si nivelul; 3 - formula comuna a barelor.



Stare	Efect principal asupra interfetei si rundei
MainMenu / Settings	Panoul corespunzator este vizibil; HUD-ul este ascuns.
Playing	Timer-ul actualizeaza sesiunea; HUD-ul citeste valorile noi.
LevelUp / Paused	Panoul este vizibil; Tick returneaza inaintea logicii jocului.
GameOver	Panoul Game Over este afisat; runda asteapta RESPAWN.

La fiecare Tick acceptat, sesiunea primeste mouse-ul in coordonate client si directia tastelor. RefreshFrame() actualizeaza HUD-ul. Daca exista niveluri in asteptare, se deschide Level Up; daca jucatorul a murit, se afiseaza Game Over. Invalidate() solicita un nou OnPaint, fara sa deseneze direct in evenimentul butonului.

5. Explicarea codului jocului

5.1 Miscarea jucatorului

GameInputController pastreaza tastele apasate intr-un HashSet<Keys>. D si A dau directia orizontala, iar S si W directia verticala. La eliberare, tasta este scoasa din set. Pozitia se actualizeaza in Tick dupa starea tastelor.

Codul 12. Directia tastelor WASD

UI/Input.cs:72-81
GameInputController.GetMovementDirection()

```
public PointF GetMovementDirection()
{
    float x = (keysDown.Contains(Keys.D) ? 1 : 0) - (keysDown.Contains(Keys.A) ? 1
    : 0);
    float y = (keysDown.Contains(Keys.S) ? 1 : 0) - (keysDown.Contains(Keys.W) ? 1
    : 0);
    // Normalize diagonals so they move at the same speed as straight lines.
    if (x != 0 && y != 0)
        return new PointF(x * 0.7071f, y * 0.7071f);
    return new PointF(x, y);
}
```

Inmultirea cu **0.7071f** normalizeaza diagonala. Player.Move() inmulteste directia cu Stats.MoveSpeed si o adauga la X si Y. ClampToArena() limiteaza pozitia la zona client; viteza initiala este 6 unitati pe actualizare, inaintea upgrade-urilor.

5.2 Sistemul de shooting

Mouse-ul este convertit prin PointToClient(Cursor.Position), astfel incat tintirea si arena folosesc aceleasi coordonate. Player.UpdateAngle() foloseste Atan2. CombatSystem.Shoot() calculeaza gura tevii, directia catre cursor si o normalizeaza. Daca mouse-ul este prea aproape, se pastreaza sensul armei pentru a evita tragerea inversa.

Codul 13. Lovitura de aproape sau crearea proiectilului

Game/Combat.cs:33-46
CombatSystem.Shoot() • partea finala a metodei

```
// Check the body-to-muzzle path so a touching enemy cannot be skipped.
Enemy? closeEnemy = FindHit(center, muzzle, true);
if (closeEnemy is not null)
    ApplyBulletHit(closeEnemy, dx, dy);
else
    game.Bullets.Add(new Bullet
    {
        X = muzzle.X - Bullet.Size / 2f, Y = muzzle.Y - Bullet.Size / 2f,
        VelocityX = dx * player.Stats.BulletSpeed,
        VelocityY = dy * player.Stats.BulletSpeed,
        Angle = MathF.Atan2(dy, dx) * 180f / MathF.PI
    });
player.ShootCooldownRemainingMs = player.Stats.FireCooldownMs;
player.RegisterShot();
```

Glontul are viteza initiala 20 si verifica intregul segment parcurs pentru prima coliziune. Damage-ul este 10-14, critica are initial 2,5% si multiplicator 1,5 sau 2. Cooldown-ul este 210 ms. Fiecare clic stang incearca un foc, fara autofoc.

5.3 Inamicii

Cele trei imagini sunt asociate cu EnemyType. Tipurile urmaresc direct jucatorul; diferentele provin din statistici, dimensiune si reactia la lovitura. Tabelul arata valorile de baza, inainte de scalarea vietii si a recompenselor.

Resursa / tip	HP	Viteza	Contact	XP de baza
Zombie_1 / ZombieFast	30	6	10	12-18
Zombie_2 / ZombieHeavy	80	3,6	20	22-32
Vampire_1 / Vampire	110	6,6	30	40-55

Codul 14. Urmarirea jucatorului

Game/Enemies.cs:138-151
EnemyManager.Update() • calculul deplasarii unui inamic

```
PointF center = enemy.GetCenter();
float dx = playerCenter.X - center.X;
float dy = playerCenter.Y - center.Y;
float distance = MathF.Sqrt(dx * dx + dy * dy);
enemy.Angle = MathF.Atan2(dy, dx) * 180f / MathF.PI;
enemy.StaggerRemainingMs = Math.Max(0, enemy.StaggerRemainingMs - elapsedMs);
float speed = enemy.Stats.Speed;
if (enemy.StaggerRemainingMs > 0)
    speed *= enemy.Stats.StaggerSpeedFactor;
if (distance > 0.001f)
{
    enemy.X += dx / distance * Math.Min(speed, distance);
    enemy.Y += dy / distance * Math.Min(speed, distance);
}
```

dx si **dy** sunt diferentele catre jucator, iar distance este lungimea directiei. Viteza scade temporar la stagger. Dupa deplasare sunt limitate pozitiile si separate corpurile suprapuse. Viata la aparitie creste cu 4% pentru fiecare nivel al jucatorului dupa nivelul 1.

5.4 Sistemul de valuri

WaveManager porneste de la valul 1. Numarul necesar este **3 + (CurrentWave − 1) × 2**, cu o aparitie programata la 650 ms. Dupa toate aparitiile si eliminarea inamicilor vii, urmeaza 2.000 ms de pauza. ZombieHeavy apare de la valul 3, iar Vampire de la valul 10; alegerea tine cont si de indicele aparitiei.

5.5 XP si Level Up

La eliminare, recompensa este acordata imediat. Enemy.RollExperienceReward() alege XP de baza si aplica +10% pentru fiecare val dupa primul, cu multiplicator maxim 4. Player.AddExperience(), din Codul 4, pastreaza surplusul si mareste pragul cu 8. GameSession pune fiecare nivel castigat in PendingLevels; interfata cere apoi cate o alegere pentru fiecare. Numai textul plutitor XP este intarziat cu 250 ms, nu recompensa.

Statistici, vindecare si randare

5.6 Upgrade-urile

UpgradeSystem pastreaza cate un contor pentru cele sapte tipuri. Fiecare alegere adauga acelasi bonus. Valorile procentuale sunt aditive; limita efectiva se aplica in PlayerStats, iar lista HUD prezinta totalul stivelor.

Upgrade normal	**Bonus pentru fiecare alegere**
Health Boost	+10 HP maxim si vindecare de pana la 10 HP
Speed Boost / Damage Boost	+3% viteza de miscare / +5% damage
Durability / Critical Training	+3 puncte procentuale rezistenta / +1 punct procentual sansa critica
Bullet Speed / Rapid Fire	+5% viteza proiectile / +4% rata de foc

Rezistenta are plafonul 60%, iar sansa critica totala 35%. Rapid Fire calculeaza cooldown-ul prin 210 / (1 + bonusul ratei de foc), fara sa scada direct procentul din timp.

5.7 Sistemul HP si bandajele

Player.TakeDamage() aplica rezistenta, pastreaza minimum 1 damage si ofera 750 ms de invulnerabilitate dupa o lovitura. La eliminarea unui inamic, conditia reala este **Random.Shared.NextDouble() < 0.10**. Bandajul vindeca pana la 20 HP, ramane pe teren la viata plina si are un hitbox de 38 × 38 pentru desenul de 28 × 28. Vindecarea produce text verde; resetarea goleste bandajele.

5.8 Randarea grafica

OnPaint() transmite Graphics la GameRenderer.Draw(), care deseneaza terenul, obiectele si personajele. Sprite-urile provin din Resources; bandajul este desenat prin forme. Jucatorul se roteste cu player.Angle in jurul pivotului sprite-ului.

Codul 15. Rotirea si desenarea unui inamic

Game/GameRenderer.cs:86-98
GameRenderer.DrawEnemies() • fragmentul de randare

```
PointF center = enemy.GetCenter();
GraphicsState state = g.Save();
// Rotating around the center keeps the sprite in place while it turns.
g.TranslateTransform(center.X, center.Y);
g.RotateTransform(enemy.Angle + 90f);

RectangleF spriteBounds = new(-size / 2f, -size / 2f, size, size);
ImageAttributes? tint = null;
if (enemy.HitFlashTimer > 0)
    tint = enemy.CriticalHitFlash ? criticalFlashAttributes : hitFlashAttributes;
DrawSprite(g, sprite, spriteBounds, tint);

g.Restore(state);
```

Save() si Restore() izoleaza transformarile. Originea se muta in centrul inamicului, se aplica rotatia, apoi DrawSprite deseneaza imaginea. Renderer-ul reutilizeaza bitmap-urile din resurse si elibereaza obiectele proprii. Invalidate() solicita un nou OnPaint.

6. Reflectie personala

Prin acest proiect am inteles mai bine diferenta dintre a construi o interfata si a programa comportamentul ei. Toolbox si Designer ma ajuta sa asez controalele, dar rezultatul devine util numai dupa ce proprietatile si evenimentele sunt conectate corect. Un exemplu simplu este PLAY: aspectul este in Designer, iar pornirea rundei ajunge in metodele din Form1.

Am invatat sa folosesc Panel pentru meniuri, Label pentru informatii si Button pentru actiuni. La HP si XP, partea vizuala m-a ajutat sa urmaresc valorile din clase. Anchor si recentrarea panourilor arata de ce o interfata nu trebuie gandita doar pentru o singura dimensiune de fereastra.

Una dintre provocari a fost organizarea codului si pastrarea unei interfete usor de modificat. Istoricul include refactorizari, iar structura actuala separa sesiunea, lupta, progresia si controlerele UI. Simplificarea functiilor si meniul Debug fac mai usoara urmarirea unei probleme: pot verifica separat valorile, evenimentul si rezultatul afisat.

7. Git si GitHub

7.1 Ce sunt Git si GitHub?

Git urmareste local versiunile fisierelor prin commit-uri. GitHub gazduieste repository-uri si permite accesul la ele prin retea. In acest proiect, origin indica repository-ul Zombie-Survivors al contului rimikkusu. Git si GitHub au roluri diferite, dar sunt folosite impreuna. [6, 7]

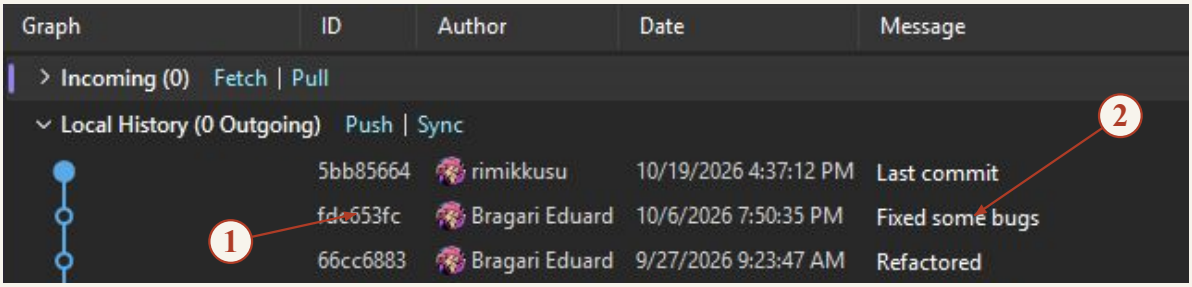
7.2 Utilizarea Git in proiect

Un flux obisnuit este **git status** pentru verificare, **git add .** pentru pregatire, **git commit -m "mesaj"** pentru salvarea versiunii si **git push** pentru trimitere. **git pull** preia si integreaza schimbarile, iar **git log** arata istoricul. Aceste comenzi sunt explicate, fara sa creez sau sa trimit un commit pentru documentatie.

7.3 Repository-ul proiectului

Commit-uri observate: **5bb8566** - „Last commit”, 19.10.2026; **fdc653f** - „Fixed some bugs”, 06.10.2026; **66cc688** - „Refactored”, 27.09.2026. Versiunea locala documentata are si modificari necomise; nu presupun ca toate se afla deja pe GitHub.

Istoric real in Visual Studio, ramura main: 1 - identificatori; 2 - mesaje de commit.



8. README si alte comenzi Git

8.1 Ce este README?

README este fisierul de prezentare al unui repository: descrie scopul, cerintele de rulare, instalarea si comenzile utilizatorului. In directorul de lucru analizat, **README.md este sters local**, deci nu prezint un continut inexistent drept documentatie actuala. Pentru o versiune viitoare ar fi utile: descrierea jocului, .NET 10 si Windows, deschiderea solutiei, comenzile WASD/mouse/ESC/F3 si structura Game/UI.

8.2 Comenzi Git utile

Comanda	Explicatie
git init	Initializeaza un repository in folderul curent.
git clone <adresa>	Copiaza un repository existent, impreuna cu istoricul sau.
git status	Afiseaza fisierele modificate, pregatite sau neurmarite.
git add .	Pregateste modificarile din directorul curent, inclusiv stergeri.
git commit -m "mesaj"	Creeaza un commit din modificarile pregatite.
git push	Trimite commit-urile catre repository-ul remote configurat.
git pull	Preia si integreaza schimbarile din ramura remote urmarita.

Comanda	Explicatie
git log / git diff	Arata istoricul / diferentele dintre versiuni sau fata de fisierele locale.

9. Concluzie

Realizarea jocului Zombie Survivors m-a ajutat sa leg notiunile de la Programarea vizuala de o aplicatie pe care o pot folosi si explica. Am construit meniuri din controale Windows Forms, am configurat proprietati si am asociat evenimentele cu metode C#. Bara HP, progresul XP si alegerea upgrade-urilor arata direct cum datele din clase ajung in interfata.

Am inteles ca o interfata clara depinde si de organizarea codului. Separarea dintre Designer, formular si clasele auxiliare imi permite sa urmaresc traseul unei actiuni: apasarea unui buton, verificarea starii, modificarea sesiunii si actualizarea ecranului. Aceasta organizare este utila atat cand adaug o functie, cat si cand rezolv o eroare.

Consider ca proiectul isi indeplineste rolul educational: include o runda de joc, progresie si meniuri functionale intr-o aplicatie desktop C#. Pentru continuare, as imbunatati documentatia repository-ului si as urmari mai atent comportamentul la diferite dimensiuni de fereastra. Cunostintele exersate aici pot fi aplicate si la alte aplicatii cu formulare, controale si evenimente.

10. Bibliografie

1. Microsoft Learn. Windows Forms overview.
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/

2. Microsoft Learn. Toolbox.
https://learn.microsoft.com/en-us/visualstudio/ide/toolbox?view=visualstudio

3. Microsoft Learn. C# documentation.
https://learn.microsoft.com/en-us/dotnet/csharp/

4. Microsoft Learn. Create a Windows Forms app in Visual Studio.
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/get-started/create-app-visual-studio

5. Microsoft Learn. Events in Windows Forms controls.
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/controls/events

6. Pro Git. Getting Started - About Version Control.
https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

7. GitHub Docs. About repositories.
https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

8. Bragari Eduard. Zombie Survivors - proiectul analizat.
https://github.com/rimikkusu/Zombie-Survivors

Material de referinta pentru structura: PAD_Studiu_Nr1.docx.pdf, furnizat pentru realizarea studiului.